SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable

movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta

Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1.AI-Powered Drones for Emergency Medicine Delivery

Introduction

AI-powered drones can deliver emergency medicines to people affected by floods. However, blocked roads, changing weather, and different travel costs make route planning difficult. Search algorithms help drones select suitable routes while considering time, cost, and safety.

i. Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search selects the next location that appears closest to the destination using a heuristic value. In this scenario, the drone uses straight-line distance to estimate how far it is from the delivery point.

Strengths:

- It can find a promising route quickly.
- It uses heuristic information to guide the search.
- It is useful when a quick decision is required.

Weaknesses:

- It ignores the actual travel cost already incurred.
- It may select a short route with strong winds or dangerous terrain.
- It does not guarantee the shortest or lowest-cost route.
- Unexpected obstacles may force the drone to change its route.

For example, the drone may choose a direct route through a flooded area because it appears close to the destination, even though another route is safer.

ii. How A* Search Handles the Scenario

A* Search considers both the actual cost of reaching a location and the estimated cost of reaching the destination.

Its evaluation function is:

\[f(n)=g(n)+h(n) \]

Where:

- \(g(n)\) = actual cost from the starting point to the current location.
- \(h(n)\) = estimated cost from the current location to the destination.
- \(f(n)\) = estimated total cost of the route through that location.

A* can consider travel time, energy consumption, and weather-related costs. It may select a slightly longer route if that route is safer and has a lower total cost.

With an appropriate admissible heuristic and suitable search conditions, A* can find an optimal route.

iii. Impact of Heuristic Accuracy

The accuracy of the heuristic affects how effectively both algorithms search for a route.

In Greedy Best-First Search, an inaccurate heuristic can direct the drone towards a route that looks close but is expensive or dangerous.

In A*, the actual cost already incurred is also considered, so the algorithm can make better cost-based decisions. However, a poor heuristic may increase the number of locations explored. If the heuristic overestimates the remaining cost, A* may lose its usual optimality guarantee.

Therefore, the heuristic should consider available information about distance, terrain, and weather.

iv. Impact of Dynamic Changes

Flooding, strong winds, and blocked paths can make a previously planned route unusable. The drone must update its map using sensor information and calculate an alternative route.

Repeatedly restarting the entire search may waste valuable time. An incremental replanning algorithm such as D* Lite can reuse earlier search information and help the drone respond more efficiently to changes.

v. Recommended Algorithm

I recommend A with dynamic replanning* for this scenario.

A* balances the actual route cost and estimated remaining cost, helping the drone choose efficient routes. A risk-aware cost function can account for weather, energy use, and dangerous areas. When new obstacles appear, a replanning method such as D* Lite can update the route.

Safety constraints should always take priority over speed. This approach offers a good balance between efficiency, safety, and real-time decision-making.

2. Smart-City Traffic Signal Optimization

Introduction

A smart city uses AI to control traffic signals and reduce waiting time, fuel consumption, and congestion. Since traffic changes throughout the day, the system must continuously evaluate and improve signal timings.

Formulating the Problem as an Optimization Problem

The objective is to minimize a combined cost:

$$ J=w_1W+w_2F+w_3C $$

Where:

- W = total vehicle waiting time.
- F = fuel consumption.

- \(C\) = traffic congestion.
- \(w_1,w_2,w_3\) = weights representing the importance of each objective.

The decision variables include green-light duration, signal phase order, and timing coordination between intersections.

The system must obey safety rules, pedestrian crossing requirements, and minimum and maximum green-light durations.

Traditional exhaustive search is inefficient because the number of possible signal combinations grows rapidly as the number of intersections increases. Evaluating every possible plan would take too much time, especially when traffic conditions are constantly changing.

i. How Hill Climbing Works and Its Limitations

Hill Climbing begins with an initial traffic signal plan and makes small changes to improve it.

For example, the system may increase the green-light duration on a busy road. If the change reduces total waiting time, it keeps the change and continues searching for improvements.

Limitations:

- It may stop at a locally optimal solution.
- It may fail to find better combinations of signal timings.
- It can struggle when several intersections need coordinated changes.
- It does not always find the best overall solution.

ii. Local Maxima, Plateaus, and Ridges

Local optimum: A signal plan appears better than nearby alternatives, but a different overall plan would reduce congestion further.

Plateau: Several signal plans have almost the same performance. The algorithm cannot easily identify which direction will improve the result.

Ridge: Improvement requires coordinated changes across multiple intersections. Changing one signal at a time may not produce an improvement, even though changing several signals together would help.

These problems are common in large traffic networks because traffic flow at one intersection affects neighbouring intersections.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing sometimes accepts a worse solution temporarily to explore other possibilities. Early in the search, it accepts such changes more readily. As the search progresses, it becomes more selective.

This helps the algorithm escape local optima and discover better signal plans.

Genetic Algorithm

A Genetic Algorithm maintains a population of possible signal plans. It evaluates their performance, selects promising plans, and creates new candidates through crossover and mutation.

This allows it to explore different parts of the search space and discover effective combinations of signal timings.

However, both approaches require careful parameter selection and must respect traffic safety constraints.

iv. Recommended Approach

I recommend a hybrid approach using Genetic Algorithms and real-time local optimization.

A Genetic Algorithm can explore different signal plans and identify good combinations across many intersections. A faster local optimization method can then adjust timings according to current traffic conditions.

The system should continuously collect traffic data, evaluate performance, and update the signal plan. This approach supports scalability and adaptability while reducing waiting time, fuel consumption, and congestion.

3. Autonomous Mars Rover Exploring Unknown Terrain

Introduction

An autonomous rover explores Mars to collect scientific samples and study the environment. It must avoid craters, rocks, and dangerous slopes while operating with limited sensor information. Since the complete map is not available beforehand, the rover must learn about its surroundings as it moves.

i. Why an Online Search Agent Is Required

Offline search plans a route using a known representation of the environment before taking action. It works well when the relevant map and available actions are already known.

However, a Mars rover cannot know every obstacle before exploration begins.

An online search agent follows a repeated process:

1. Observe the surroundings.
2. Select an action.
3. Move to a new location.
4. Collect new information.
5. Update its internal map.
6. Plan the next action.

For example, if the rover discovers a crater that was not previously known, it can update its map and choose another route.

Therefore, online search is more suitable because it allows the rover to learn and make decisions during exploration.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability means that the rover cannot observe the complete environment at once. Rocks may hide obstacles, camera images may be unclear, and sensor readings may contain errors.

The rover must make decisions using incomplete information. It should also estimate how reliable its observations are.

Other challenges include:

- Unknown terrain and steep slopes.
- Limited battery power.
- Communication delays with Earth.
- Uncertain wheel traction.
- Sensor errors and localization uncertainty.
- Newly discovered hazards.

These challenges make safe navigation difficult. The rover must consider uncertainty and avoid routes that could damage its equipment.

iii. How the Rover Builds and Updates Its Internal Model

The rover uses cameras, distance sensors, and movement measurements to build a map of its surroundings.

Its internal model may contain:

- Known obstacles and safe areas.
- Terrain slopes and surface conditions.
- The rover's estimated position.
- Unexplored regions.
- Uncertain locations that require further inspection.

After moving, the rover compares sensor observations with its existing map. If it discovers a new obstacle, it updates the map and recalculates its route.

This process helps the rover improve its understanding of the environment over time.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Uninformed Search	Informed Search	Online Search

Information used	No goal-directed heuristic	Uses a heuristic	Learns through observations
Environment knowledge	Usually assumes a known search space	Usually uses a known search model	Can operate with an incomplete map
Examples	BFS, UCS	A*, Greedy Best-First	Online exploration with repeated replanning
Main advantage	Simple search rules	Can guide search towards a goal	Adapts to newly discovered information
Main limitation	May explore many states	Depends on model and heuristic quality	Must make decisions with incomplete information

Online search is not necessarily a replacement for informed or uninformed search. It can combine either approach with continuous observation and replanning.

v. Recommended Approach

I recommend an online search agent combined with A and risk-aware replanning*.

The rover should maintain a local map, use sensors to detect hazards, and calculate routes using distance, energy consumption, terrain difficulty, and safety.

Whenever new information becomes available, it should update its map and re-plan if necessary. It should move cautiously through uncertain regions and avoid areas where the risk is too high.

This approach is suitable because it combines adaptability, efficient planning, and safety.

4. University Exam Timetable as a Constraint Satisfaction Problem (CSP)

Introduction

A university must prepare an exam timetable that avoids student clashes, uses available examination halls efficiently, and respects faculty availability and other academic requirements.

A Constraint Satisfaction Problem (CSP) is suitable because it involves assigning values to variables while satisfying a set of constraints.

i. Variables, Domains, and Constraints

A CSP consists of three main components: variables, domains, and constraints.

1. Variables

For every examination, define variables such as:

- Examination time slot.
- Examination hall.
- Assigned invigilator, if required.

For example, for a Mathematics exam, the variables may be its time slot and examination hall.

2. Domains

A domain contains the possible values that can be assigned to a variable.

For example:

- Time domain: 9 AM, 11 AM, 2 PM.
- Hall domain: Hall A, Hall B, Hall C.

The domains must include only available time slots and suitable examination halls.

3. Constraints

The timetable must satisfy the following rules:

- No student clashes: A student cannot have two examinations at the same time.
- Hall capacity: The selected hall must accommodate all students taking the examination.
- Hall availability: A hall cannot host two examinations simultaneously.
- Subject order: If one subject must be completed before another, their time slots must follow that order.
- Faculty availability: Examinations and invigilation duties must fit staff availability.
- Special accommodations: Students requiring accessible facilities must be assigned appropriate halls and arrangements.

The university may also have soft constraints, such as avoiding too many examinations for a student on the same day.

ii. How Backtracking Search Works

Backtracking search assigns values to variables one at a time and checks whether each assignment satisfies the constraints.

The process is:

1. Select an examination.
2. Choose a possible time slot and hall.
3. Check whether the assignment conflicts with existing assignments.
4. If it is valid, continue with the next examination.
5. If no valid assignment is possible, return to an earlier decision and try another option.
6. Continue until a complete valid timetable is found.

For example, if a student already has an examination at 9 AM, the system cannot schedule another examination for that student at the same time.

Backtracking systematically explores possible assignments until it finds a valid solution or determines that the current choices cannot produce one.

iii. How Constraint Propagation Improves Efficiency

Constraint propagation reduces the number of choices available before the search reaches a conflict.

One useful technique is forward checking.

Whenever a time slot is assigned to an examination, the system removes that time slot from the domains of other examinations that share students.

Similarly, when a hall is assigned to an examination, that hall is removed from the available choices for other examinations occurring at the same time.

This helps detect conflicts early and reduces unnecessary searching.

Other techniques, such as maintaining arc consistency, can remove additional impossible values from variable domains.

iv. Real-World Challenges and the Best Strategy for Scalability

Real-world examination scheduling can become difficult because of:

- A large number of courses and students.
- Limited examination halls.
- Faculty timetable changes.
- Special accommodation requirements.
- Last-minute course additions or cancellations.
- Conflicting student preferences.

As the problem grows, the number of possible combinations increases significantly.

I recommend backtracking search combined with constraint propagation and intelligent variable ordering.

The Minimum Remaining Values (MRV) heuristic selects the variable with the fewest legal choices first. This helps the system handle the most restricted examinations early.

For larger universities, the system can divide the problem into manageable groups while checking conflicts between groups. After finding a valid timetable, local optimization can improve student convenience and room utilization.

This strategy improves efficiency while maintaining the required constraints.

5. Strategic Game AI for Real-Time Decision Making: Minimax and Alpha-Beta Pruning

Introduction

A gaming company wants to develop an AI opponent that can compete with human players. The AI must examine possible moves, predict opponent responses, and make decisions within a strict time limit.

Minimax and Alpha-Beta pruning are useful techniques for strategic decision-making, particularly in adversarial games.

i. How Minimax Models the Problem

Minimax represents possible game moves as a search tree.

It considers two types of players:

- MAX: The AI attempts to maximize its evaluation score.
- MIN: The opponent attempts to minimize the AI's evaluation score.

The algorithm explores possible moves and opponent responses. It then works backward from the evaluated game states to select the move with the best expected worst-case outcome.

For example, if the AI has three possible moves, it evaluates how the opponent could respond to each move. It then selects the move that provides the strongest result against the opponent's best response.

Minimax is most directly applicable to turn-based, two-player, zero-sum games. Real-time strategy games may require adaptations for simultaneous actions, hidden information, and very large action spaces.

ii. Computational Challenges of Large Game Trees

The main problem with Minimax is the large number of possible game states.

If each state has \(b\) possible actions and the search depth is \(d\), the approximate number of nodes examined is:

[O(b^d) \]

As the branching factor and search depth increase, the number of states grows rapidly.

This creates several challenges:

- High computation time.
- Large memory requirements in some implementations.
- Difficulty evaluating every possible move.
- Risk of missing the real-time decision deadline.
- Difficulty predicting long-term consequences.

Therefore, exploring the complete game tree is often impractical.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning improves Minimax by skipping branches that cannot affect the final decision.

It maintains two values:

- Alpha (\(\alpha\)): The best score MAX can guarantee so far.
- Beta (\(\beta\)): The best score MIN can guarantee so far.

When a branch proves that a player already has a better alternative, the algorithm can stop exploring that branch.

Example:

Suppose MAX has already found a move with a score of 8. While examining another move, MIN finds a response that limits its score to 5.

Since MAX already has a move worth 8, it will not choose the alternative that can be limited to 5. The algorithm can prune the remaining unnecessary branches of that alternative.

The main advantages are:

- Fewer nodes need to be evaluated.
- Deeper searches may become possible within the same time limit.
- The final decision remains the same as Minimax for the same fully searched tree.
- Good move ordering can significantly improve pruning.

In the worst case, Alpha-Beta pruning may offer little improvement, but with good move ordering it can greatly reduce the search effort.

iv. Design an Evaluation Function and Justify the Factors

An evaluation function estimates how favourable a game state is for the AI.

A simple example is:

[E(s)=4M+3R+2P+5T-6H \]

Where:

- \(M\) = relative military strength.
- \(R\) = relative resource advantage.
- \(P\) = strategic position or map control.
- \(T\) = threat posed to important enemy objectives.
- \(H\) = danger to the AI's own base and important units.

The numerical weights are illustrative and should be adjusted through testing.

Why these factors matter:

- Military strength: Helps estimate the AI's ability to attack and defend.
- Resources: Measures the ability to produce units and maintain an economy.
- Position: Rewards control of useful areas and strategic locations.

- Enemy threats: Encourages progress towards important objectives.
- Defensive risk: Penalizes moves that leave the AI's base or units vulnerable.

The evaluation function should also consider unit survival, mission objectives, and time-sensitive threats. In games with hidden information, uncertain estimates should be treated carefully.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

I recommend Iterative Deepening Minimax with Alpha-Beta pruning.

Iterative deepening searches the game tree at increasing depths. It first completes a shallow search and then moves to deeper levels while time remains.

If the time limit is reached, the AI can use the best move from its last completed search.

Additional improvements include:

- Move ordering: Examine promising moves first to increase pruning.
- Transposition tables: Store results for previously evaluated game states.
- Time limits: Stop searching before the decision deadline.
- Efficient evaluation: Use a fast function to estimate game-state quality.
- Selective search: Spend more computation on important battles or threats.

For a real-time strategy game, the AI should always return a legal action within the available time. It may not always find the globally best move, but this approach offers a practical balance between decision quality, efficiency, and responsiveness.

Conclusion

Artificial Intelligence provides different methods for solving problems involving search, optimization, uncertainty, and decision-making.

In emergency medicine delivery, A* with dynamic replanning can help drones choose efficient and safer routes. In smart-city traffic management, Genetic Algorithms and other optimization techniques can improve traffic flow across many intersections. For Mars exploration, online search allows a rover to learn about unknown terrain and adjust its plans. Constraint Satisfaction Problems help universities create valid examination timetables, while Minimax with Alpha-Beta pruning enables game AI to make strategic decisions efficiently.

The best algorithm depends on the problem's requirements, available information, computational resources, and safety constraints. In real-world applications, combining algorithms with c